

РАБОТА С БИБЛИОТЕКАМИ В PYTHON.



Что такое библиотеки в Python?

Библиотеки в Python - это наборы функций, классов и модулей, которые упрощают решение различных задач.

Когда вы подключаете библиотеку в Python, вы получаете доступ к её методам и классам, которые значительно упрощают выполнение специфических задач. Это сэкономит вам время и силы, а также повысит читаемость вашего кода.

Для подключения библиотеки в Python используется конструкция `import`:

```
import название_библиотеки
```

Затем вы можете использовать все функции и классы, доступные в этой библиотеке.

Библиотека **math**

Библиотека **math** включает множество полезных математических функций, которые пригодятся при решении задач, связанных с числами. Рассмотрим некоторые из них.

```
1  import math
2
3  # Квадратный корень числа
4  print(math.sqrt(16))  # 4.0
5
6  # Вычисление факториала
7  print(math.factorial(5))  # 120
8
9  # Синус угла (в радианах)
10 print(math.sin(math.pi / 2))  # 1.0
11
12 # Возведение в степень
13 print(math.pow(2, 3))  # 8.0
```

**Можете записать только 4 и 13 строки, остальные вам не понадобятся*

Библиотека **random**

Библиотека **random** используется для генерации случайных чисел. Это полезно, когда нужно случайным образом выбрать элементы, например, для игр, тестов или случайных событий.

```
1 import random
2
3 # Случайное целое число от 1 до 10
4 print(random.randint(1, 10))
5
6 # Случайный элемент из списка (списки мы разберём чуть позже)
7 fruits = ["яблоко", "банан", "вишня"]
8 print(random.choice(fruits))
9
10 # Перемешивание списка
11 random.shuffle(fruits)
12 print(fruits)
```

Способы импорта библиотек в Python

В Python существует несколько способов импорта библиотек и модулей. Рассмотрим основные методы.

- Обычный импорт

Самый простой способ импортировать библиотеку — использовать команду **import**

Этот способ позволяет получить доступ ко всем функциям и классам библиотеки, используя имя библиотеки в качестве префикса.

```
1 import имя_библиотеки
2 имя_библиотеки.функция
```

Способы импорта библиотек в Python

- Импорт с псевдонимом

Если название библиотеки слишком длинное или если вы хотите использовать короткое имя для удобства, вы можете задать псевдоним (alias):

```
import math as m  
import random as r
```

Теперь вы можете использовать псевдонимы для обращения к библиотекам.

```
m.sqrt(16)  
r.randint(1, 10)
```

Способы импорта библиотек в Python

- Импорт отдельных функций или классов

Если вам нужно только несколько функций или классов из библиотеки, можно импортировать их напрямую, без использования префикса с именем библиотеки. Это особенно полезно, если библиотека содержит множество функций, а вам нужно только несколько.

```
from math import sqrt, pow
from random import randint
```

Теперь можно использовать функции напрямую.

```
sqrt(16)
pow(2, 3)
randint(1, 10)
```

Способы импорта библиотек в Python

- Импорт всего содержимого библиотеки

В случае, если вам нужно использовать все функции и классы из библиотеки, можно импортировать её полностью.

```
from math import *  
from random import *
```

Этот способ не рекомендуется использовать в больших проектах, так как он может привести к конфликту имен, если в разных библиотеках есть одноимённые функции. Лучше использовать первый или второй способ, чтобы избежать таких проблем.

Способы импорта библиотек в Python

- Импорт модулей из пакетов

Python поддерживает иерархию пакетов, то есть библиотеки могут содержать несколько модулей. Чтобы импортировать определённый модуль из пакета, нужно указать путь к нему:

```
1 from package_name.module_name import function_name
2 from datetime import datetime
```

Библиотека Turtle

С помощью библиотеки **turtle** можно рисовать на экране, управляя «черепашкой», которая оставляет за собой след.

Подключаем библиотеку

```
import turtle
```

Настройка черепашки

Изменение формы черепашки

Черепашка может выглядеть по-разному. Для этого используем команду **shape()**. Вот возможные формы:

arrow	стрела	
turtle	черепашка (по умолчанию)	
circle	круг	
square	квадрат	
triangle	треугольник	
classic	классический вид	

```
turtle.shape("turtle")
```

Настройка черепашки

Изменение цвета

Вы можете изменить цвет, которым черепашка будет рисовать линии, с помощью команды **color()**:

```
turtle.color("green")
```

	AliceBlue		bisque3		CadetBlue		CornflowerBlue		DarkGrey		DarkSeaGreen
	AntiqueWhite		bisque4		CadetBlue1		cornsilk		DarkKhaki		DarkSeaGreen1
	AntiqueWhite1		black		CadetBlue2		cornsilk1		DarkMagenta		DarkSeaGreen2
	AntiqueWhite2		BlanchedAlmond		CadetBlue3		cornsilk2		DarkOliveGreen		DarkSeaGreen3
	AntiqueWhite3		blue		CadetBlue4		cornsilk3		DarkOliveGreen1		DarkSeaGreen4
	AntiqueWhite4		blue1		chartreuse		cornsilk4		DarkOliveGreen2		DarkSlateBlue
	aquamarine		blue2		chartreuse1		cyan		DarkOliveGreen3		DarkSlateGray
	aquamarine1		blue3		chartreuse2		cyan1		DarkOliveGreen4		DarkSlateGray1
	aquamarine2		blue4		chartreuse3		cyan2		DarkOrange		DarkSlateGray2
	aquamarine3		BlueViolet		chartreuse4		cyan3		DarkOrange1		DarkSlateGray3
	aquamarine4		brown		chocolate		cyan4		DarkOrange2		DarkSlateGray4
	azure		brown1		chocolate1		DarkBlue		DarkOrange3		DarkSlateGrey
	azure1		brown2		chocolate2		DarkCyan		DarkOrange4		DarkTurquoise
	azure2		brown3		chocolate3		DarkGoldenrod		DarkOrchid		DarkViolet
	azure3		brown4		chocolate4		DarkGoldenrod1		DarkOrchid1		DeepPink
	azure4		burlywood		coral		DarkGoldenrod2		DarkOrchid2		DeepPink1
	beige		burlywood1		coral1		DarkGoldenrod3		DarkOrchid3		DeepPink2
	bisque		burlywood2		coral2		DarkGoldenrod4		DarkOrchid4		DeepPink3
	bisque1		burlywood3		coral3		DarkGray		DarkRed		DeepPink4
	bisque2		burlywood4		coral4		DarkGreen		DarkSalmon		DeepSkyBlue

Настройка черепашки

Изменение скорости

Черепашка может двигаться с разной скоростью от 1 (медленно) до 10 (быстро)

С помощью команды **speed()**

```
turtle.speed(5)
```

Двигаем черепашку и рисуем

forward(distance) — движение вперёд на указанное количество пикселей.

backward(distance) — движение назад на указанное количество пикселей.

left(angle) — поворот на указанный угол влево.

right(angle) — поворот на указанный угол вправо.

```
9  turtle.forward(30) →
10 turtle.left(90) ↶
11 turtle.forward(30) ↑
12 turtle.left(90) ↷
13 turtle.forward(30) ←
14 turtle.left(90) ↵
15 turtle.forward(30) ↓
16
17 turtle.done() # Завершаем работу
```

Красный — двигаемся вперёд

Зеленый - угол

Использование циклов для удобства

```
1 import turtle
2
3 turtle.shape("turtle")
4 turtle.color("green")
5 turtle.speed(5)
6
7 for i in range(4):
8     turtle.forward(30)
9     turtle.left(90)
10
11 turtle.done()
```

Использование переменных для удобства

```
40 import turtle
41
42 turtle.shape("turtle")
43 turtle.color("green")
44 turtle.speed(5)
45
46 size = 50
47
48 for i in range(4):
49     turtle.forward(size)
50     turtle.left(90)
51
52 turtle.done()
```

Рисуем окружности **circle()**

Черепашка может рисовать не только прямые линии, но и окружности. Для этого используется метод **circle()**, в который нужно передать радиус окружности:

```
58 import turtle
59
60 turtle.shape("turtle")
61 turtle.color("blue")
62 turtle.speed(5)
63
64 # Рисуем окружность с радиусом 50 пикселей
65 turtle.circle(50)
66
67 turtle.done()
```

Перемещение без рисования

Иногда бывает нужно **перемещать** черепашку, но не рисовать при этом **линию**. Для этого используем команды:

- **penup()** — поднимает перо, и черепашка перестает рисовать.
- **pendown()** — опускает перо, и черепашка снова начинает рисовать.

```
import turtle
```

```
turtle.shape("turtle")
```

```
turtle.color("red")
```

```
turtle.speed(5)
```

```
# Двигаем черепашку без рисования
```

```
turtle.penup() # Поднимаем перо
```

```
turtle.forward(100) # Двигаем черепашку вперёд
```

```
turtle.pendown() # Опускаем перо
```

```
for i in range(4):
```

```
    turtle.forward(50)
```

```
    turtle.left(90)
```

```
turtle.done()
```

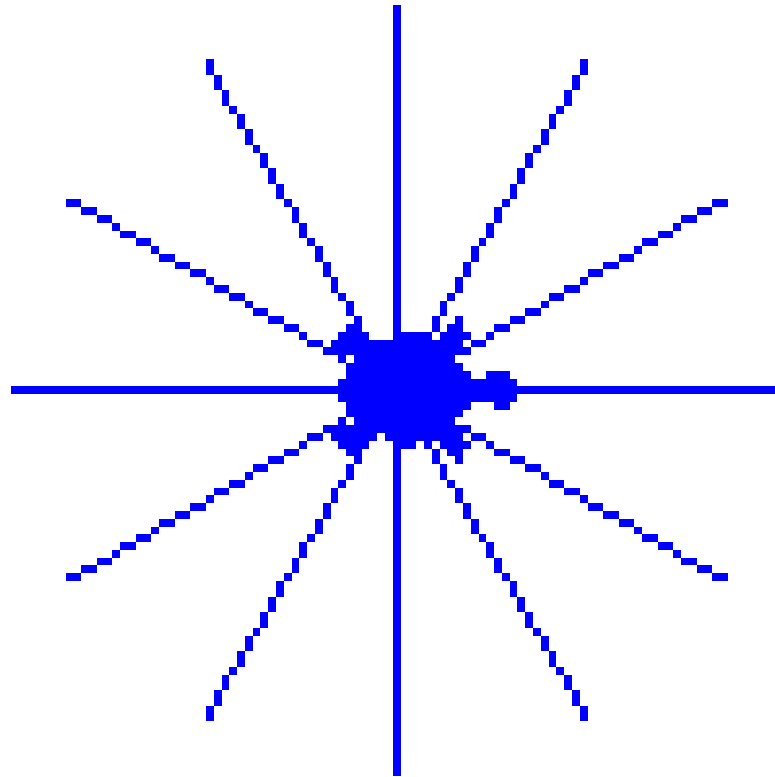
рисовать при этом **линию**.

рисовать.

инает рисовать.

Немного практики

Давайте нарисуем простую снежинку



```
1 import turtle
2 turtle.shape("turtle")
3 turtle.color("blue")
4 turtle.speed(1)
5
6 for i in range(12):
7     turtle.forward(50) # рисуем линию
8     # Возвращаемся в исходное положение
9     turtle.left(180)
10    turtle.penup() # Поднимаем перо
11    turtle.forward(50)
12    turtle.pendown() # Опускаем перо
13
14    turtle.left(30) # отступ
15 turtle.done()
```

СПИСКИ В PYTHON — РАБОТА С КОЛЛЕКЦИЯМИ ДАННЫХ



Что такое списки?

Списки — это специальные структуры, которые позволяют хранить сразу несколько значений в одной переменной.

```
names = ["Аня", "Боря", "Вика"]
```

Здесь `names` — это список, содержащий три строки: «Аня», «Боря» и «Вика». Список обозначается квадратными скобками `[]`. Внутри скобок перечислены элементы (значения), через запятую.

Где применяются списки?

Списки — это универсальный инструмент, который используется:

- для хранения списка учеников
- результатов теста
- шагов в алгоритме
- игровых объектов (например, список врагов)
- хранения введённых пользователем данных

Как устроен список?

Списки могут содержать:

- ничего(пустой список)
- числа
- строки
- перемешанные типы данных
- даже другие списки

```
my_list = [] # пустой список
numbers = [1, 2, 3, 4, 5] # список из чисел
words = ["apple", "banana", "cherry"] # список из строк
mixed = [1, "hello", True, 3.14] # смешанный список

nested = [[1, 2], [3, 4]]
```

Что такое индекс?

Чтобы получить доступ к отдельному элементу списка, мы используем индекс. **Индекс** — это номер элемента в списке. В Python нумерация начинается с **нуля**!

```
fruits = ["яблоко", "банан", "вишня"]  
  
print(fruits[0])    # яблоко  
  
print(fruits[1])    # банан  
  
print(fruits[2])    # вишня
```

Это исторически сложилось из математики и работы с памятью в компьютерах. Первый элемент находится на 0-м шаге от начала. Это удобно для программистов и экономит ресурсы.

Синтаксис работы с индексами

```
список[номер_элемента]
```

- список[0] — первый элемент
- список[1] — второй элемент
- и так далее

Вы также можете использовать отрицательные индексы:

```
print(fruits[-1]) # последний элемент: вишня  
print(fruits[-2]) # предпоследний элемент: банан
```

Что можно делать со списками? **append()**

Списки обладают множеством методов — специальных встроенных команд, которые позволяют изменять, сортировать, добавлять и удалять элементы. Рассмотрим их по порядку с объяснениями.

1. **append()** Добавляет элемент в конец списка.

```
fruits.append("груша")  
print(fruits)  # ['яблоко', 'банан', 'груша']
```

append() — это как «приклеить» новый элемент в конец списка. Он ничего не возвращает, просто изменяет список.

Что можно делать со списками? **insert()**

2. **insert()** Вставляет элемент в нужное место по индексу.

```
fruits.insert(1, "вишня")  
print(fruits) # ['яблоко', 'вишня', 'банан', 'груша']
```

insert(1, «вишня») вставит «вишня» перед элементом с индексом 1.
Все элементы сдвинутся вправо.

Что можно делать со списками? **remove()**

3. **remove()** Удаляет первое вхождение элемента.

```
fruits.remove("банан")  
print(fruits) # ['яблоко', 'вишня', 'груша']
```

remove() ищет первое совпадение и удаляет его. Если элемента нет, будет ошибка.

Что можно делать со списками? **pop()**

4. **pop()** Удаляет элемент по индексу и возвращает его.

```
last_fruit = fruits.pop()
print(last_fruit)    # груша
print(fruits)        # ['яблоко', 'вишня']
```

Без аргумента **pop()** удаляет последний элемент. Если указать индекс — удалит элемент с этим индексом.

Что можно делать со списками? **len()**

5. **len()** Функция len() возвращает количество элементов в списке.

```
print(len(fruits))    # 2
```

Очень полезно, когда нужно узнать длину списка, например, в цикле.

Что можно делать со списками?

6. **sort()** и **reverse()**

Сортируют и переворачивают список.

```
numbers = [3, 1, 4, 2]
numbers.sort()    # сортировка по возрастанию
print(numbers)    # [1, 2, 3, 4]

numbers.reverse() # разворот списка
print(numbers)    # [4, 3, 2, 1]
```

Цикл for — перебор элементов списка

Если у нас список из 100 элементов, и нам нужно вывести все — писать 100 строк с `print(list[0])`, `print(list[1])` и так далее — это нереально.

Для этого есть циклы — чтобы автоматически повторять действия.

```
fruits = ["яблоко", "банан", "груша"]  
  
for fruit in fruits:  
    print(fruit)
```

Python по очереди берёт каждый элемент из списка `fruits` и помещает его в переменную `fruit`, затем выполняет код внутри цикла.

Цикл for с индексами

Иногда нужно знать не только значение, но и индекс.

```
for i in range(len(fruits)):
    print(f"Элемент {i}: {fruits[i]}")
```

len(fruits) — длина списка (кол-во элементов)

range(len(fruits)) — генератор чисел от 0 до длины минус 1

Мы используем *i* как индекс для доступа к элементу

Спасибо за внимание!

